Learning Lean 4

Joshua W. Kelly

2026-7-18

Abstract

Working notes and exercises from learning the Lean 4 theorem prover, following *An Introduction to Lean 4* by Cosme Llópez and Gong. Covers basic syntax and natural-deduction proofs, paired with a runnable Lean library.

Contents

1 Basic Syntax

1.1 Basic syntax

The runnable version of this chapter lives in `LearningLean/BasicSyntax.lean`.

1.1.1 Comments

```
-- This is a single-line comment.

/-
This is a multi-line comment.
Here is another line.
-/
```

1.1.2 Inspecting terms and types with `#check`

`#check` asks Lean for the type of an expression without evaluating it.

```
#check true          -- Bool
#check 42            -- Nat
#check "Hello, World!" -- String
#check Type          -- Type 1
#check Nat           -- Type
#check ['h', 'e', 'l', 'l', 'o'] -- List Char
```

1.1.3 Printing definitions with `#print`

`#print` shows how something is defined.

```
#print Bool
```

`Type` itself cannot be printed with `#print`, because it is a type rather than a definition.

1.1.4 Definitions and functions

```
def pi : Float := 3.1415926

#check pi

-- An anonymous function (lambda) taking two natural numbers.
#check λ (a b : Nat) => a + b
```

2 Natural Deduction

2.1 Natural deduction

The Lean proofs for this chapter live in `LearningLean/NaturalDeduction.lean`, inside the `Exercises` namespace.

2.1.1 5.5 Reduction to the absurd. $P \Rightarrow Q,\ \neg Q \vdash \neg P$

This is a very useful technique. Validity of $P \Rightarrow Q,\ \neg Q \vdash \neg P$ is proved with:

1	$P \Rightarrow Q$	
2	$\neg Q$	
3	$\quad P$	H
4	$\quad Q$	E⇒ 1,3
5	$\quad \neg Q$	IT 2
6	$\neg P$	I¬ 3,4,5

The sub-derivation (lines 3–5) discharges the hypothesis P: assuming P, [⇒ -elimination] on lines 1 and 3 gives Q, while line 2 reiterates $\neg Q$. The contradiction lets us introduce $\neg P$ (¬-introduction over lines 3, 4, 5).

In Lean this is theorem `T55`, where `¬P` is definitionally `P → False`,[1] so the whole proof is just function composition:

```
theorem T55 (h1 : P → Q) (h2 : ¬Q) : ¬P :=
  (h2 ∘ h1)
```

2.1.2 Conjunction introduction. $P,\ P \Rightarrow Q \vdash P \wedge Q$

Theorem `T51` builds the pair `⟨proof of P, proof of Q⟩` directly, using the anonymous-constructor notation `⟨_, _⟩`:

```
theorem T51 (h1 : P) (h2 : P → Q) : P ∧ Q :=
  ⟨h1, h2 h1⟩
```

[1] This is the standard definition in Lean's core library: `Not p` unfolds to `p → False`, so a proof of `¬P` is exactly a function turning a proof of `P` into a proof of `False`.

3 Linear Algebra

3.1 Linear algebra

A scratch chapter for linear-algebra notes — enough structure to flesh out later, and enough variety (numbered equations, a table, admonitions, and interleaved Lean) to show the book template in action. The runnable code lives in `LearningLean/LinearAlgebra.lean`.

> **Note**
>
> This chapter is a work in progress. Sections marked **To flesh out** are placeholders.

3.1.1 Vectors

A vector in R^n is an ordered list of n real numbers. We write column vectors as $\vec{u} = (u_1, \ldots, u_n)$ with each $u_i \in R$.

The **dot product** of $\vec{u}, \vec{v} \in R^n$ is the scalar

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^{n} u_i v_i.$$

For a concrete computation, take $\vec{u} = (1, 2, 3)$ and $\vec{v} = (4, 5, 6)$. Then by equation (1),

$$\vec{u} \cdot \vec{v} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32.$$

In Lean, a vector is a function `Fin n → ℝ`, and `![…]` is notation for one. The dot product and the matrix operations below follow the spellings used in Mathlib[1:2]

```
def u : Fin 3 → ℝ := ![1, 2, 3]
def v : Fin 3 → ℝ := ![4, 5, 6]

#check u ⬝ᵥ v      -- `⬝ᵥ` is Mathlib's dot product, of type ℝ
```

3.1.2 Matrices

An $m \times n$ matrix $A \in R^{m \times n}$ is a rectangular array of scalars. For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \in R^{2 \times 2}.$$

[2] Mathlib writes the dot product as `⬝ᵥ` (`\cdotv`) rather than reusing `·`, to keep it distinct from scalar multiplication.

The **matrix–vector product** $A\vec{x}$ takes a vector $\vec{x} \in R^n$ to a vector in R^m. With $\vec{x} = (5, 6)$:

$$A\vec{x} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}.$$

A few operations and their Mathlib spellings:

Operation	Math	Lean / Mathlib
Dot product	$\vec{u} \cdot \vec{v}$	`u ⬝ᵥ v`
Matrix–vector product	$A\vec{x}$	`A *ᵥ x`
Matrix product	AB	`A * B`
Transpose	A^T	`Aᵀ`

```
def A : Matrix (Fin 2) (Fin 2) ℝ := !![1, 2; 3, 4]

#check A *ᵥ ![5, 6]   -- matrix-vector product, of type `Fin 2 → ℝ`
```

3.1.3 Linear maps

A **linear map** $T : R^n \to R^m$ is a function satisfying $T(\alpha\vec{u} + \vec{v}) = \alpha\, T(\vec{u}) + T(\vec{v})$ for all scalars α and vectors $\vec{u}, \vec{v}$. Every such map is represented by a matrix, and matrix multiplication corresponds to composition of the maps.

> **Note**
>
> **To flesh out:** state and prove that `Matrix.mulVec A` is a `LinearMap`, and connect it to `Matrix.toLin`.

3.1.4 A worked computation

To flesh out. A step-by-step example — e.g. solving $A\vec{x} = \vec{b}$ by row reduction — narrated alongside the corresponding Lean definitions, so the computation and its formalization sit side by side.

3.1.5 Exercises

> **Note**
>
> **To flesh out.** Add exercises here, e.g.:
>
> 1. Show the dot product is symmetric: $\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{u}$.
> 2. Prove $(A^\mathsf{T})^\mathsf{T} = A$ in Lean using `Matrix.transpose_transpose`.

References

1. The mathlib Community. The Lean Mathematical Library. in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)* 367–381 (2020). doi:10.1145/3372885.3373824.